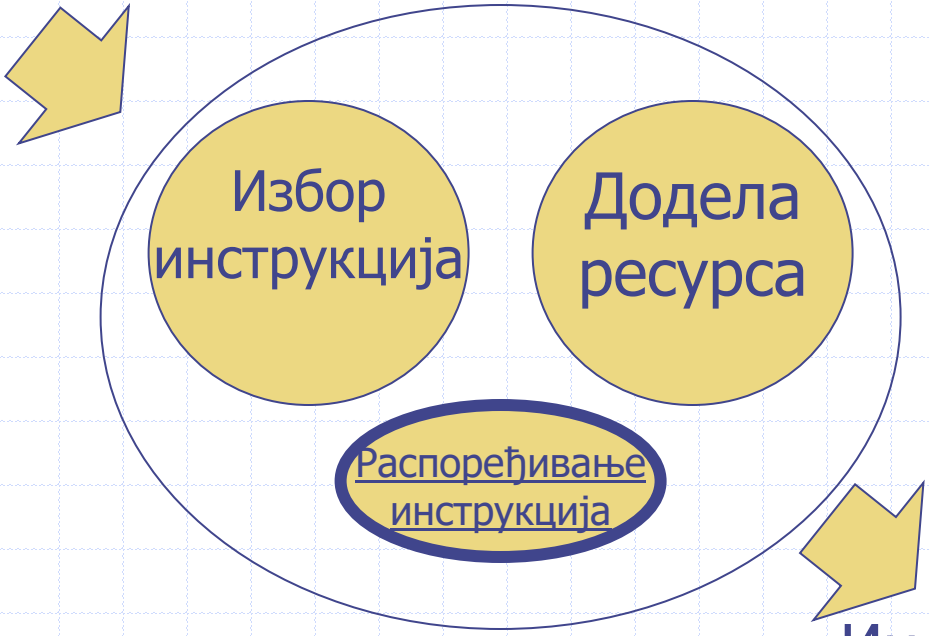
Поједностављени поглед на задњи део компајлера

Међурепрезентација
(Међујезик – IR)



Инструкције циљне архитектуре

Поједностављени поглед на задњи део компајлера

Потпуни асемблерски код (без осврта на проточну структуру)



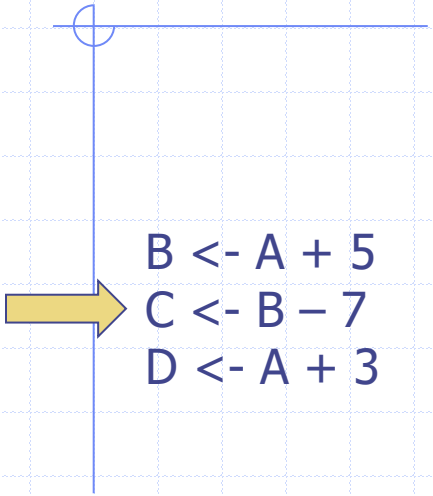
Потпуно функционална асемблерски код



B <- A + 5
C <- B – 7
D <- A + 3

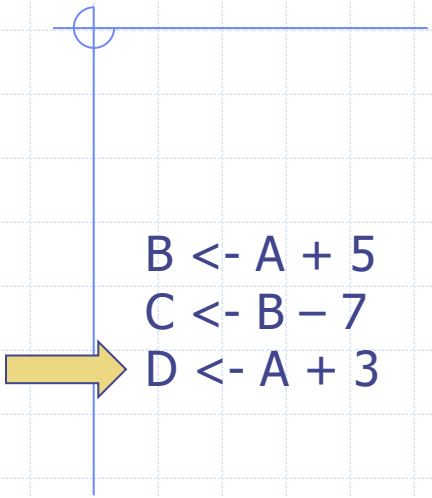
Ако је ово почетно стање, колико ће бити C?

A = 10; B = 55; C = 97



B <- A + 5
C <- B – 7
D <- A + 3

A = 10; **B = 15**; C = 97



A = 10; B = 15; **C = 8**

B <- A + 5

Учитавање	Учитај A
Сабирање	
Уписивање	



B <- A + 5

Учитавање	
Сабирање	Додај 5
Уписивање	



B <- A + 5

Учитавање	
Сабирање	
Уписивање	Упиши B

B <- A + 5
C <- B – 7
D <- A + 3

A = 10; B = 55; C = 97

Учитавање	Учитај A	B <- A + 5
Сабирање		NOP
Уписивање		NOP

C <- B – 7
D <- A + 3

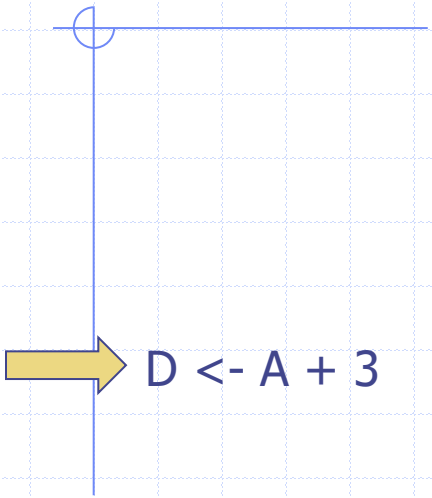
A = 10; B = 55; C = 97

Учитавање	Учитај B	C <- B - 7
Сабирање	Додај 5	B <- A + 5
Уписивање		NOP

C <- B – 7
D <- A + 3

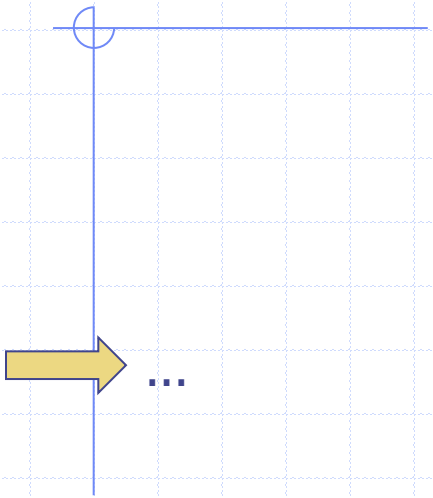
A = 10; B = 55; C = 97

Учитавање	Учитај B	C <- B - 7
Сабирање	Додај 5	B <- A + 5
Уписивање		NOP



A = 10; **B = 15**; C = 97

Учитавање	Учитај A	D <- A + 3
Сабирање	Одузми 7	C <- B - 7
Уписивање	Упиши B	B <- A + 5



A = 10; B = 15; **C = 48**

Учитавање	...	...
Сабирање	Додај 3	D <- A + 3
Уписивање	Упиши C	C <- B - 7

Два главна разлога за распоређивање

- Проточна структура
  - Различите фазе обраде различитих инструкција се извршавају у паралели
- Широка инструкциона реч
  - Више инструкција (операција) се извршавају у паралели

```
x0=xmem[i1]
i1+=1
y1=ymem[i4]
i4-=1
a0=a1+b1
b0=b1+a1
```

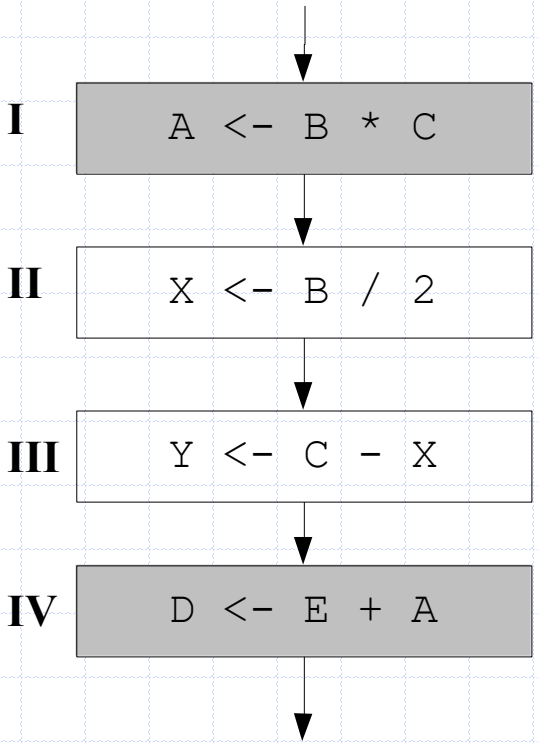
```
x0=xmem[i1]; i1+=1; y1=ymem[i4]; i4-=1; a0=a1+b1; b0=b1+a1
```

Хардверска подршка проточној структури

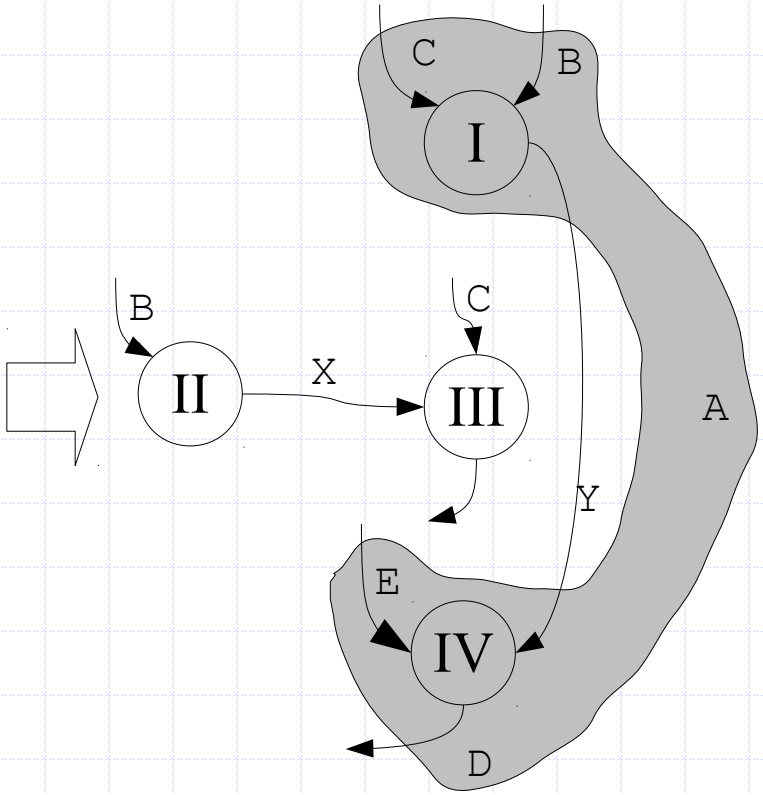
- Оквирно три нивоа:

1. Скривена проточна структура
   - Хардвер води рачуна и о аспекту ваљаности и о аспекту оптимизације (динамичко распоређивање)
2. Аутоматски дистанцер (уметање балончића)
   - Хардвер води рачуна само о аспекту ваљаности
3. Експлицитно распоређивање
   - Програмер води рачуна о проточној структури

Илустрација односа тока управљања и тока података



Граф тока управљања



Граф тока података

Врсте зависности

```
1.X <- 5 - Z
2.Y <- X * 7
3.X <- 2
```

- Читање после писања (RAW – true)
  - Излаз из И1 је улаз у И2
- Писање после читања (WAR – anti)
  - Излаз из И2 је улаз у И1
- Писање после писања (WAW – output)
  - Излаз из И1 је излаз и из И2

Аспект ваљаности

- Довољан размак између инструкција
  - Код проточне структуре
- Исправност инструкције са једном операцијом
  - Код широке инструкционе речи

Динамичко распоређивање (у процесору)

За овај улаз:

```
1. A <- B * C
2. X <- B / 2
3. Y <- C - X
4. D <- E + A
```

Ће се заправо извршити ово:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

Аутоматски дистанцер

За овај улаз:

```
1. A <- B * C
2. X <- B / 2
3. Y <- C - X
4. D <- E + A
```

Ће се заправо извршити ово:

```
1. A <- B * C
2. X <- B / 2
   NOP
3. Y <- C - X
4. D <- E + A
```

5 циклуса, уместо 4

Експлицитно распоређивање

За овај улаз:

```
1. A <- B * C
2. X <- B / 2
3. Y <- C - X
4. D <- E + A
```

Ће се заправо извршити ово:

```
1. A <- B * C
2. X <- B / 2
3. Y <- C - X
4. D <- E + A
```

Али ће резултат бити другачији!

Динамичко распоређивање (у процесору)

За овај улаз:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

Ће се заправо извршити ово:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

Аутоматски дистанцер

За овај улаз:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

Ће се заправо извршити ово:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

Експлицитно распоређивање

За овај улаз:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

Ће се заправо извршити ово:

```
2. X <- B / 2
1. A <- B * C
3. Y <- C - X
4. D <- E + A
```

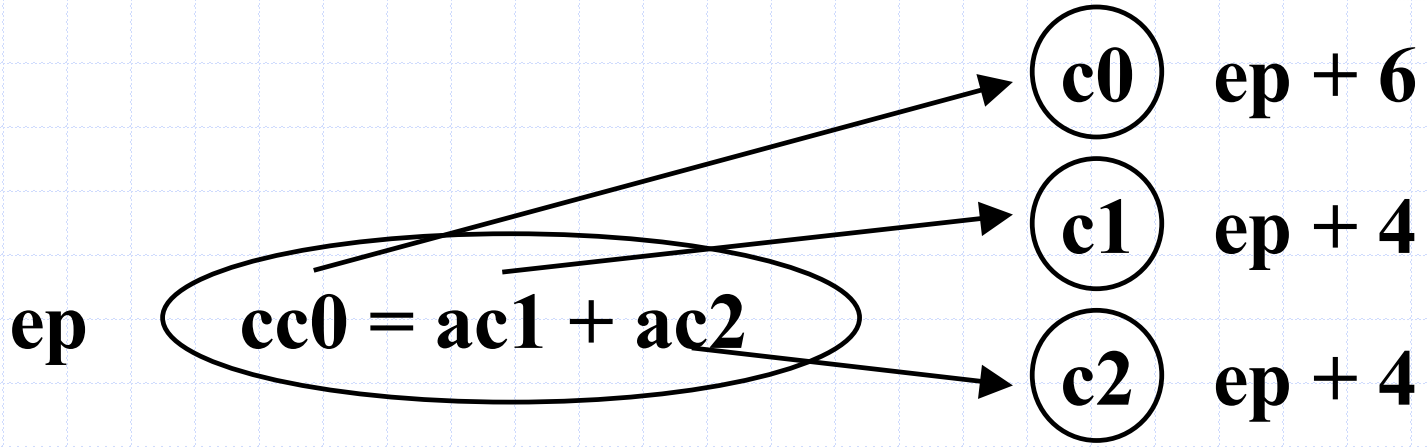
Фаза припреме кода за проточну обраду

- Да би се генерисани код припремио за проточну обраду, у којој су фазе обраде суседних инструкција међусобно преклопљене, између суседних инструкција се мора уметнути потребан број NOP инструкција.
- Тиме се спречава појава конфликата на магистрали и обезбеђује да се упис у неки објекат (ресурс) у текућој инструкцији заврши пре његове употребе у наредној инструкцији.

Тачка приступа (ap) и тачка извршења (ep)

- Тачка извршења је једнака индексу инструкције у низу инструкција (прва инструкција у низу има ep=0).
- Тачка приступа операнду, ap, се добија тако што се на ep дода редни број фазе проточне обраде у којој се приступа операнду.
- ep је везан за целу инструкцију и зависи од њене позиције
- ap је везан за операнд (не зависи од позиције инструкције)

Пример израчунавања ар на основу ер



Аспект оптимизације

- Ниво парова инструкција
- Ниво основног блока
- Ниво секвенцних група (трагова)

Аспект оптимизације

- Две групе приступа:
  - Слагајући распоређивач
  - Перколациони распоређивач

Слагајући распоређивач

- Инструкције (операције) из основног блока се убаце у листу.
- Листа се уреди по неком приоритету.
- Основни блок се реконструише постепено, пребацивањем једне по једне инструкције из листе у основни блок.

Слагајући распоређивач

- Неке хеуристике за редослед слагања инструкција (операција):
  - Предност имају операције које користе ресурсе које већ не користе већ убачене операције
  - Предност имају инструкције које могу да се ставе што даље од инструкција са којима су у вези
  - Предност имају инструкције које имају више зависних инструкција